

Experiment 2

Siddhant Bajaj

Aim: Create a Blockchain using Python

Theory:

Blockchain is a distributed and immutable ledger that records data in a sequence of linked blocks. Each block contains a timestamp, proof (from Proof-of-Work), and the hash of the previous block, ensuring integrity and security.

In this program:

1. **Block Creation** – A block is generated using `create_block()` with a proof and previous hash.
2. **Proof-of-Work (PoW)** – Implemented to mine a block by solving a cryptographic puzzle (finding a hash with leading zeros).
3. **Hashing** – SHA-256 algorithm ensures data immutability.
4. **Validation** – `is_chain_valid()` checks that every block correctly references the previous hash and satisfies PoW conditions.
5. **Flask Web API** – Routes `/mine_block`, `/get_chain`, and `/is_valid` allow mining, fetching the chain, and verifying blockchain integrity through a browser or API client.

This program runs on a local server and simulates mining a blockchain without a network of nodes, making it an ideal introductory model to understand blockchain basics.

Code:

1. Importing Required Libraries

- **datetime** → to timestamp each block
- **hashlib** → to generate SHA-256 hashes
- **json** → to encode blocks as JSON for hashing
- **Flask** → to run the blockchain API server

2. Blockchain Class – Core Logic

This contains everything needed to:

- create blocks
- store the chain
- implement Proof of Work
- validate chain integrity

3. Flask Web API

Provides endpoints:

- `/mine_block` → mine a new block
- `/get_chain` → fetch full blockchain
- `/is_valid` → verify blockchain

Program and Output

Program 1:

```
!pip install flask
import datetime
import hashlib
import json
from flask import Flask, jsonify, request

# -----
# Blockchain Class
# -----

class Blockchain:
    def __init__(self):
        # List to store all blocks
        self.chain = []

        # List to store pending transactions
        self.transactions = []

        # Create Genesis Block
        self.create_block(proof=1, previous_hash='0')

# -----
```

```

# Create a new transaction
# -----
def create_transactions(self, sender, receiver, amount):
    transaction = {
        'sender': sender,
        'receiver': receiver,
        'amount': amount
    }

    # Add transaction to pending transactions
    self.transactions.append(transaction)
    return transaction

# -----
# Create a new block
# -----
def create_block(self, proof, previous_hash):
    block = {
        'index': len(self.chain) + 1,
        'timestamp': str(datetime.datetime.now()),
        'proof': proof,
        'previous_hash': previous_hash,
        'transactions': self.transactions.copy()
    }

    # Add block to blockchain
    self.chain.append(block)
    # Remove transactions after mining
    self.transactions.clear()
    return block

# -----
# Get the previous block
# -----
def get_previous_block(self):
    return self.chain[-1]

```

```

# -----
# Proof of Work
# -----

def proof_of_work(self, previous_proof):
    new_proof = 1
    while True:
        # Cryptographic puzzle
        hash_operation = hashlib.sha256(
            str(new_proof ** 2 - previous_proof ** 2).encode()
        ).hexdigest()
        # Required: 3 leading zeros
        if hash_operation.startswith('000'):
            return new_proof
        new_proof += 1

```

```

# -----
# Hash a block
# -----

```

```

def hash(self, block):
    encoded_block = json.dumps(
        block,
        sort_keys=True
    ).encode()
    return hashlib.sha256(encoded_block).hexdigest()

```

```

# -----
# Check whether blockchain is valid
# -----

```

```

def is_chain_valid(self, chain):
    if len(chain) == 0:
        return False
    previous_block = chain[0]

```

```

    block_index = 1
    while block_index < len(chain):
        block = chain[block_index]
        # Check previous hash
        if block['previous_hash'] != self.hash(previous_block):
            return False
        # Check Proof of Work
        previous_proof = previous_block['proof']
        proof = block['proof']
        hash_operation = hashlib.sha256(
            str(proof ** 2 - previous_proof ** 2).encode()
        ).hexdigest()

        if not hash_operation.startswith('000'):
            return False
        previous_block = block
        block_index += 1
    return True

# -----
# Flask Application
# -----

app = Flask(__name__)
# Create Blockchain object
blockchain = Blockchain()

# -----
# API: Create Transaction
# -----

@app.route('/add_transaction', methods=['POST'])
def add_transaction():
    data = request.get_json()
    sender = data.get('sender')
    receiver = data.get('receiver')
    amount = data.get('amount')

```

if sender is None or receiver is None or amount is None:

```
    return jsonify({  
        'message': 'Please provide sender, receiver and amount.'  
    }), 400
```

transaction = blockchain.create_transactions(

sender,

receiver,

amount

)

return jsonify({

'message': 'Transaction added successfully.',

'transaction': transaction,

'pending_transactions': blockchain.transactions

}), 201

-----

API: Mine a Block

-----

@app.route('/mine_block', methods=['GET'])

def mine_block():

Do not mine if transaction list is empty

if len(blockchain.transactions) == 0:

return jsonify({

'message': 'No transactions available. Add transactions before mining.'

}), 400

Get previous block

previous_block = blockchain.get_previous_block()

Get previous proof

previous_proof = previous_block['proof']

Find Golden Nonce

proof = blockchain.proof_of_work(previous_proof)

```

# Get previous block hash
previous_hash = blockchain.hash(previous_block)
# Create new block
block = blockchain.create_block(
    proof,
    previous_hash
)
return jsonify({
    'message': 'Congratulations! Block mined successfully.',
    'block': block,
    'golden_nonce': proof
}), 200

# -----
# API: Get Complete Blockchain
# -----

@app.route('/get_chain', methods=['GET'])
def get_chain():

    response = {
        'chain': blockchain.chain,
        'length': len(blockchain.chain)
    }

    return jsonify(response), 200

# -----
# API: Check Blockchain Validity
# -----

@app.route('/is_valid', methods=['GET'])
def is_valid():

```

```

    valid = blockchain.is_chain_valid(
        blockchain.chain
    )
    if valid:
        response = {
            'message': 'All good. The Blockchain is valid.'
        }
    else:
        response = {
            'message': 'The Blockchain is not valid.'
        }
    return jsonify(response), 200
# -----
# Run Flask Application
# -----

import threading

def run_flask():
    app.run(host='0.0.0.0', port=5001)

flask_thread = threading.Thread(target=run_flask)
flask_thread.daemon = True
flask_thread.start()
print("Flask server started successfully!")

```

Output

Flask server started successfully!

* Serving Flask app '__main__'

* Debug mode: off

INFO:werkzeug:WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.

* Running on all addresses (0.0.0.0)

* Running on <http://127.0.0.1:5001>

* Running on <http://172.28.0.12:5001>

Program 2:

```
!pip install pyngrok -q  
from pyngrok import ngrok  
# Add your ngrok authentication token  
ngrok.set_auth_token("3ILlAMng4Aq7WLi4mYCS04aKcIh_69V93mRpmvMmc9seCCnLX")  
# Connect ngrok to your Flask server  
public_url = ngrok.connect(5001)  
print("Blockchain API URL:")  
print(public_url)
```

Output

Blockchain API URL:

NgrokTunnel: "<https://veneering-never-playpen.ngrok-free.dev> " -> "<http://localhost:5001>"

Program 3:

```
url = "https://veneering-never-playpen.ngrok-free.dev/add_transaction"
```

```
transactions = [  
    {  
        "sender": "Siddhant",  
        "receiver": "Bob",  
        "amount": 200  
    },  
    {  
        "sender": "Bob",  
        "receiver": "Dave",  
        "amount": 500  
    },  
    {  
        "sender": "Dave",  
        "receiver": "Eve",  
        "amount": 100  
    },  
    {  
        "sender": "Eve",
```

```
        "receiver": "Alice",
        "amount": 300
    }
]
```

for transaction in transactions:

```
    response = requests.post(url, json=transaction)
```

```
    print("Transaction:", transaction)
```

```
    print("Status:", response.status_code)
```

```
    print(response.json())
```

```
    print()
```

Output

INFO:werkzeug:127.0.0.1 - - [24/Aug/2026 06:37:33] "POST /add_transaction HTTP/1.1" 201 -

INFO:werkzeug:127.0.0.1 - - [24/Aug/2026 06:37:33] "POST /add_transaction HTTP/1.1" 201 -

Transaction: {'sender': 'Siddhant', 'receiver': 'Bob', 'amount': 200}

Status: 201

```
{'message': 'Transaction added successfully.', 'pending_transactions': [{'amount': 200, 'receiver': 'Bob',
'sender': 'Siddhant'}], {'amount': 200, 'receiver': 'Bob', 'sender': 'Siddhant'}], 'transaction': {'amount': 200,
'receiver': 'Bob', 'sender': 'Siddhant'}}
```

Transaction: {'sender': 'Bob', 'receiver': 'Dave', 'amount': 500}

Status: 201

```
{'message': 'Transaction added successfully.', 'pending_transactions': [{'amount': 200, 'receiver': 'Bob',
'sender': 'Siddhant'}], {'amount': 200, 'receiver': 'Bob', 'sender': 'Siddhant'}, {'amount': 500, 'receiver':
'Dave', 'sender': 'Bob'}], 'transaction': {'amount': 500, 'receiver': 'Dave', 'sender': 'Bob'}}
```

INFO:werkzeug:127.0.0.1 - - [24/Aug/2026 06:37:33] "POST /add_transaction HTTP/1.1" 201 -

INFO:werkzeug:127.0.0.1 - - [24/Aug/2026 06:37:33] "POST /add_transaction HTTP/1.1" 201 -

Transaction: {'sender': 'Dave', 'receiver': 'Eve', 'amount': 100}

Status: 201

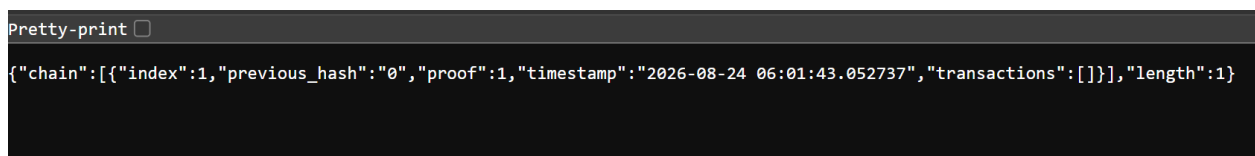
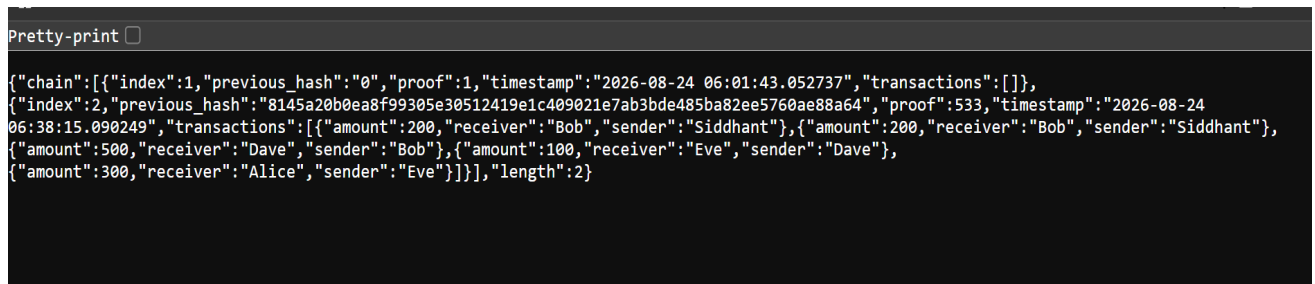
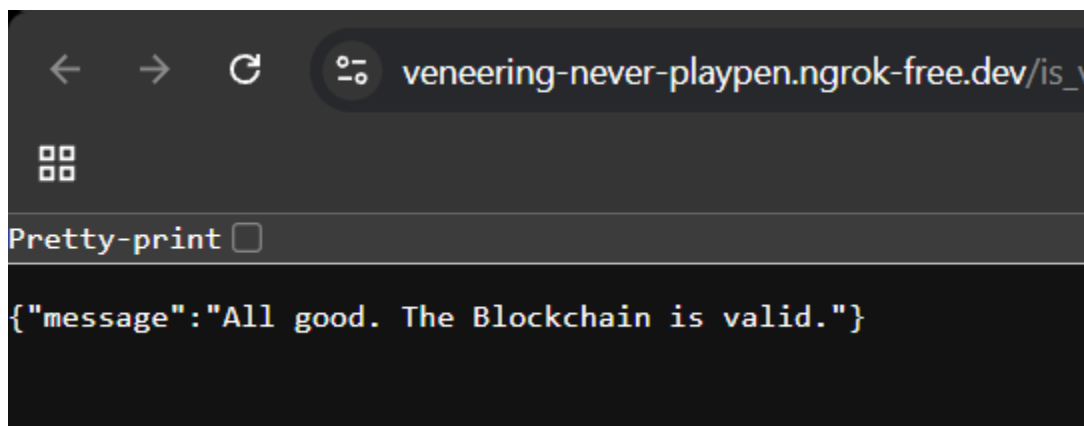
```
{'message': 'Transaction added successfully.', 'pending_transactions': [{'amount': 200, 'receiver': 'Bob',
'sender': 'Siddhant'}], {'amount': 200, 'receiver': 'Bob', 'sender': 'Siddhant'}, {'amount': 500, 'receiver':
```

```
'Dave', 'sender': 'Bob'}}, {'amount': 100, 'receiver': 'Eve', 'sender': 'Dave'}], 'transaction': {'amount': 100, 'receiver': 'Eve', 'sender': 'Dave'}}
```

```
Transaction: {'sender': 'Eve', 'receiver': 'Alice', 'amount': 300}
```

```
Status: 201
```

```
{'message': 'Transaction added successfully.', 'pending_transactions': [{'amount': 200, 'receiver': 'Bob', 'sender': 'Siddhant'}, {'amount': 200, 'receiver': 'Bob', 'sender': 'Siddhant'}, {'amount': 500, 'receiver': 'Dave', 'sender': 'Bob'}, {'amount': 100, 'receiver': 'Eve', 'sender': 'Dave'}, {'amount': 300, 'receiver': 'Alice', 'sender': 'Eve'}], 'transaction': {'amount': 300, 'receiver': 'Alice', 'sender': 'Eve'}}
```



Conclusion:

We successfully created a basic blockchain using Python and Flask that supports block mining, chain retrieval, and validity checks. Proof-of-Work ensures security by making block creation computationally intensive, while cryptographic hashing guarantees immutability. The implementation demonstrates core

blockchain principles—decentralization, immutability, and consensus—in a simplified environment, providing a strong foundation for building more advanced blockchain systems.